

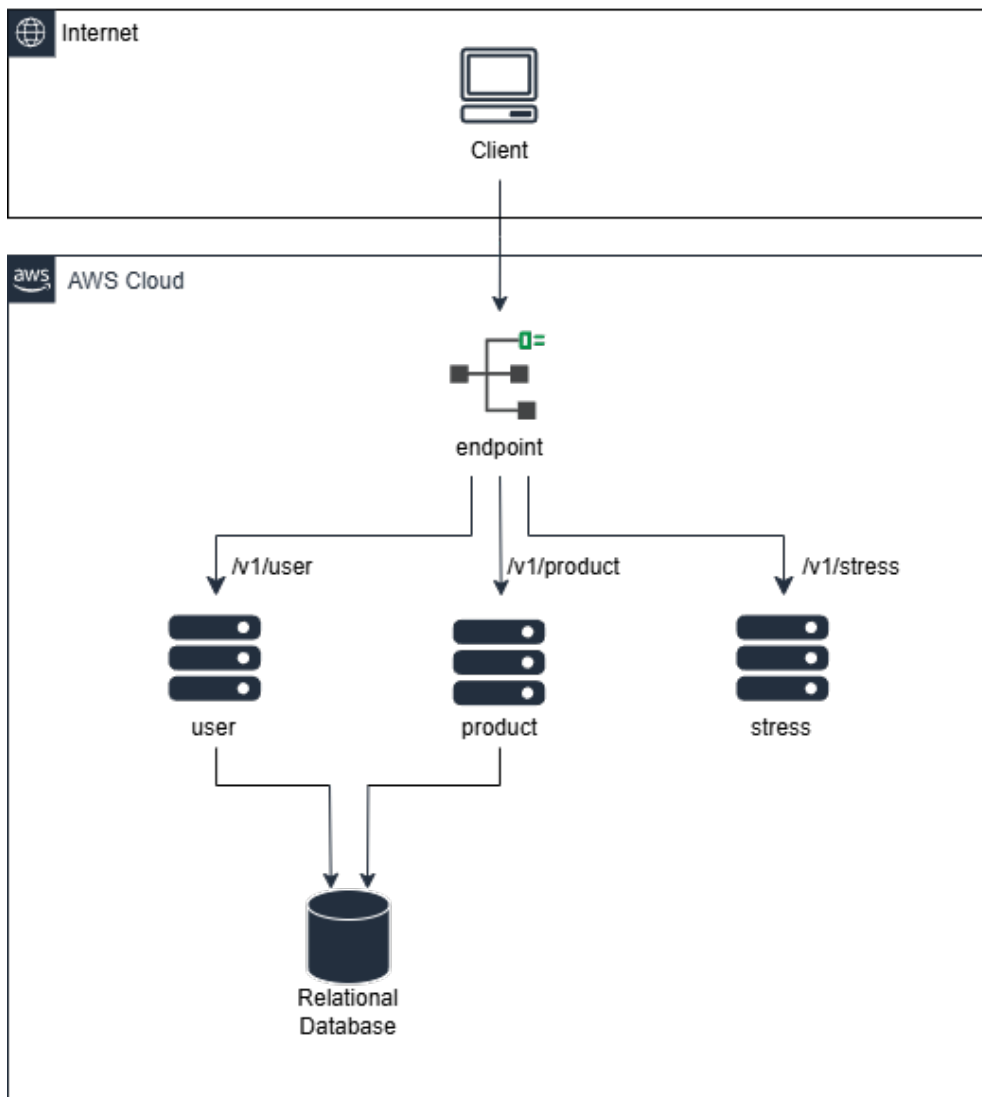
2026년도 전국기능경기대회

직 종 명	클라우드컴퓨팅	과 제 명	System operation	과제번호	제 3과제
경기시간	3시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

개발된 어플리케이션을 배포하여 사용자들에게 서비스를 제공하는 시스템을 구축하고 운영을 위한 모니터링 시스템을 구축합니다. 목표로 설정한 서비스 수준을 만족하도록 트래픽을 처리해야 하며, 구축한 시스템에 발생할 수 있는 장애와 오류를 빠르게 감지하여 최적의 사용자 경험을 제공하도록 시스템을 운영해야 합니다. 동시에 최대한 저비용으로 시스템을 운영할 수 있도록 합니다.

다이어그램



필수 Software Stack

AWS	개발언어/프레임워크
- VPC - EKS, EC2, ECR - RDS - S3	- Golang / Gin

사용 가능 Software Stack

AWS	개발언어/프레임워크
- ELB, API Gateway - CloudFront, R53 - Cloudwatch - WAF	- Docker

2. 선수 유의사항

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 선수의 계정에는 비용제한이 존재하며, 이보다 더 높게 과금될 시 계정 사용이 불가능할 수 있습니다.
- 6) 문제에 제시된 괄호박스 <>는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 7) 과제 종료 시 진행 중인 테스트를 모두 종료하여 서버에 부하가 발생 하지 않도록 합니다.
- 8) 별도 언급이 없는 경우, ap-northeast-2 리전에 리소스를 생성하도록 합니다.
- 9) 1페이지의 다이어그램은 구성을 추상적으로 표현한 그림으로, 세부적인 구성은 아래의 요구사항을 만족시킬 수 있도록 합니다. (ex. 서브넷이 2개 이상 존재할 수 있습니다.)
- 10) 모든 리소스의 이름, 태그, 변수는 대소문자를 구분합니다.
- 11) 문제에서 주어지지 않는 값들은 AWS Well-Architected Framework 6 pillars를 기준으로 적절한 값을 설정해야 합니다.
- 12) 불필요한 리소스를 생성한 경우 감점의 요인이 될 수 있습니다.
(e.g. 미사용 EC2, VPC 추가, 다른 리전 리소스 생성 등)
- 13) DB 및 모든 리소스에 대해 정해진 인스턴스 타입, CPU, Memory 크기와 대수를 지키지 않으면 불이익이 있을 수 있습니다. (미사용도 모두 포함)
- 14) 모든 시간은 KST(UTC+9) 타임존을 사용합니다.
- 15) 컴퓨팅 타입은 EC2 인스턴스만 사용하도록 합니다. 어떠한 목적으로든 Fargate와 Lambda를 사용할 수 없습니다.

3. 관계형 데이터베이스

user, product 어플리케이션은 관계형 데이터베이스를 사용하며 읽기, 쓰기 작업이 모두 가능해야 합니다. 어플리케이션은 MySQL Community 엔진을 지원합니다. DB 인스턴스는 최소한으로 운영합니다.

- DB identifier : apdev-rds-instance
- Deployment options: Multi-AZ DB instance
- DB instance class: db.t3.micro
- Storage type: General Purpose SSD (gp3)
- Engine : MySQL Community 8.0

아래의 명령어를 참고하여 테이블을 생성할 수 있습니다.

```
CREATE TABLE user (  
    id            VARCHAR(255)    NOT NULL,  
    username      VARCHAR(255)    NOT NULL,  
    email         VARCHAR(255)    NOT NULL,  
    PRIMARY KEY (id),  
    UNIQUE KEY uk_username (username)  
);  
  
CREATE TABLE product (  
    id            VARCHAR(255)    NOT NULL,  
    name          VARCHAR(255)    NOT NULL,  
    price         FLOAT(8)        NOT NULL,  
    image_path     VARCHAR(500)    DEFAULT NULL,  
    PRIMARY KEY (id)  
);
```

정상적인 동작을 위해서 load_user.dump 파일을 사용하여 데이터를 삽입해야 하며, 삽입한 데이터는 임의로 수정하거나 삭제하지 않도록 합니다. 발생하는 트래픽 외 임의의 데이터를 삽입하면 성능 저하가 생길 수 있으므로 주의하도록 합니다. 어플리케이션 동작과 트래픽 패턴에 맞게 테이블 구조 재설계가 필요할 수 있습니다.

5. 웹 어플리케이션

user, product, stress 총 3 개의 어플리케이션이 있습니다. 제공된 binary는 x86기반 EC2 의 Amazon Linux 2023 에서 빌드하고 동작을 확인하였습니다. go version은 go1.22.2 linux/amd64 입니다. 어플리케이션 실행 시 바인딩되는 포트는 TCP/8080 입니다. 모든 어플리케이션은 access log를 stdout, stderr로 출력합니다. 모든 어플리케이션은 변조 방지를 위해 Request 요청에 requestid, uuid 쿼리스트링이 추가됩니다. 정상적인 요청에 대해 Request 및 Response를 변조하여 불이익을 받지 않도록 합니다. 채점 시 사용되는 응답시간 및 코드는 모두 클라이언트 도착 기준입니다.

- user

사용자 정보를 관리하는 API를 제공합니다. 어플리케이션의 동작은 아래를 참고합니다. user API는 최소한의 가용성을 위해 5초 이하의 응답시간을 보장하고자 하며, 0.2초 이하의 응답시간을 서비스 수준 목표로 설정하였습니다. 해당 기준으로 서비스 수준을 측정하고 어플리케이션을 운영하도록 합니다.

Path	Method	Request (example)	Response code
/v1/user	POST	{ "requestid":"999999999999", "uuid":"7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729", "username":"dbdump500001", "email":"dbdump500001@example.org", }	201
/v1/user	GET	?email=dbdump500001@example.org &requestid=999999999999 &uuid=7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729	200
/healthcheck	GET	-	200

user 어플리케이션은 실행 시 아래의 리눅스 환경변수를 읽어 관계형 데이터베이스에 연결합니다.

Environment Key	Data Type	Value	etc
MYSQL_USER	string	db 접근 권한을 가지는 유저의 이름	required
MYSQL_PASSWORD	string	db 접근 권한을 가지는 유저의 암호	required
MYSQL_HOST	string	db 읽기/쓰기가 가능한 주소 (IP or DNS 이며 엔진명 삽입금지)	required
MYSQL_PORT	integer	db 읽기/쓰기가 가능한 주소의 포트번호	required
MYSQL_DBNAME	string	논리적인 데이터베이스 이름 (dev)	required

- product

제품 정보를 관리하는 API를 제공합니다. 어플리케이션의 동작은 아래를 참고합니다. product API는 제품 생성(POST 요청) 이후 간헐적인 제품 이미지 변경 외 정보 변동은 이뤄지지 않으며 동일한 id에 대한 요청이 빈번하게 발생할 것으로 예상됩니다. product API는 최소한의 가용성을 위해 5초 이하의 응답시간을 보장하고자 하며, 0.2초 이하의 응답시간을 서비스 수준 목표로 설정하였습니다. 해당 기준으로 서비스 수준을 측정하고 어플리케이션을 운영하도록 합니다.

Path	Method	Request (example)	Response code
/v1/product	POST	{ "requestid": "999999999999", "uuid": "7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729", "id": "dbdump500001", "name": "dbdump500001", "price": 1234, }	201
/v1/product	GET	?id=dbdump50001 &requestid=999999999999 &uuid=7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729	200
/v1/product	PUT	{ "requestid": "999999999999", "uuid": "7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729", "id": "dbdump500001" } include an small image file.	200
/healthcheck	GET	-	200

product 어플리케이션은 실행 시 아래의 리눅스 환경변수를 읽어 관계형 데이터베이스에 연결합니다.

Environment Key	Data Type	Value	etc
MYSQL_USER	string	db 접근 권한을 가지는 유저의 이름	required
MYSQL_PASSWORD	string	db 접근 권한을 가지는 유저의 암호	required
MYSQL_HOST	string	db 읽기/쓰기가 가능한 주소 (IP or DNS 이며 엔진명 삽입금지)	required
MYSQL_PORT	integer	db 읽기/쓰기가 가능한 주소의 포트번호	required
MYSQL_DBNAME	string	논리적인 데이터베이스 이름 (dev)	required

- stress

리소스에 부하를 발생시키는 데모 API를 제공합니다. 어플리케이션의 동작은 아래를 참고합니다. stress API는 최소한의 가용성을 위해 5초 이하의 응답시간을 보장하고자 하며, 1초 이하의 응답시간을 서비스 수준 목표로 설정하였습니다. 해당 기준으로 서비스 수준을 측정하고 어플리케이션을 운영하도록 합니다.

Path	Method	Request (example)	Response code
/v1/stress	POST	{ "requestid": "999999999999", "uuid": "7c5a3c6a-758f-4bc5-9bdf-3e573a0ad729", "length": 256 }	201
/healthcheck	GET	-	200

6. S3

제품의 이미지를 제공하기 위해 오브젝트 스토리지인 S3 를 구성합니다. 버킷 이름은 자유롭게 명명하도록 하며, 어플리케이션을 배포한 리전에 버킷을 생성하는 것을 권장합니다. product 어플리케이션에서 이미지 업로드가 가능하도록 설정해야 합니다. 해당 버킷에 저장되어 있는 오브젝트들은 어플리케이션과 동일한 엔드포인트로 인터넷 사용자들에게 제공되어야 합니다. 최종적으로 인터넷 사용자는 /images/<object path> 경로 GET 요청으로 이미지를 다운로드 받을 수 있어야 합니다.

경기 중 1시간 뒤부터 발생하는 트래픽과 product 어플리케이션의 동작으로 이미지 업로드가 발생되며, 이미지 다운로드 시나리오의 모의 트래픽이 함께 발생합니다. 엔드포인트 설정에 대한 자세한 내용은 **7. 트래픽 처리 내용**을 참고합니다.

- 예시: (이미지 경로) /product50001.jpg

(다운로드 경로) <endpoint>/images/product50001.jpg

이미지 다운로드는 최소한의 가용성을 위해 5초 이내의 응답시간을 보장하고자 하며, 서비스 수준 목표 또한 동일하게 설정하였습니다. 해당 기준으로 서비스 수준을 측정하고 정적 콘텐츠를 제공할 수 있도록 합니다.

7. 트래픽 처리

어플리케이션을 배포하고 외부 사용자들에게 서비스를 제공할 수 있도록 시스템을 구축합니다. 컨테이너를 활용해야 하며 컨테이너 오케스트레이션 툴로는 EKS를 사용합니다. ECS를 사용할 수 없습니다. 컨테이너 이미지 레포지토리로는 ECR을 사용할 수 있으며 컨테이너 호스팅을 위한 컴퓨팅 리소스는 EC2를 사용합니다.

EC2 인스턴스는 t3.medium 타입만을 사용하도록 하며, 트래픽 처리를 위해 최소한의 리소스만을 사용하도록 합니다. 다른 인스턴스 타입을 사용하거나 과도한 리소스를 사용할 경우 감점의 원인이 될 수 있습니다.

트래픽 처리를 위해 사용자에게 제공되는 엔드포인트를 하나로 단일화하도록 합니다. 경기

시작 1시간 뒤부터 트래픽이 발생하며 채점 플랫폼에 입력한 엔드포인트로 선수의 시스템에 주입됩니다. 경기 중 발생하는 트래픽을 목표 서비스 수준에 만족하도록 처리하세요. 모니터링 환경과 로깅 솔루션을 구축하여 로그 분석, 트래픽 패턴 분석, 시스템 개선, 시스템 장애/오류 감지 및 대처를 수행해야 합니다. 본인의 엔드포인트를 타인에게 노출하여 불이익을 받는일이 없도록 합니다. 채점 플랫폼에 입력하는 형식은 프로토콜 및 주소(IP/DNS)이며 경로를 적어선 안됩니다.

- 정상 예시: <https://example.org>

- 잘못된 예시(1): example.org (프로토콜 누락)

- 잘못된 예시(2): <https://example.org/v1/> (경로 기입)

서비스 중단 없이 안정적으로 서비스를 제공할 수 있도록 가용성 확보합니다. 이를 고려하여 시스템을 설계하고 구축한 뒤 운영할 수 있도록 합니다.

사용자에게 제공하는 엔드포인트로의 비정상적인 요청은 Block 하도록 하며, 403 응답코드를 내려주도록 합니다. 단, 제공하는 API 외의 요청은 404 응답코드를 내려주도록 합니다.

예제)

- [/v1/user](#) 으로의 비정상 요청: 403 응답코드

- [/v1/none](#) 으로의 비정상 요청: 404 응답코드